

[Open in app](#)[Get started](#)

Published in Data.gov.sg Blog



Open Government Products

[Follow](#)Dec 1, 2016 · 8 min read · [Listen](#)

Save



How the Circle Line rogue train was caught with data

Text: Daniel Sim | Analysis: Lee Shangqian, Daniel Sim & Clarence Ng

Singapore's MRT Circle Line was hit by a spate of mysterious disruptions in recent months, causing much confusion and distress to thousands of commuters.

Like most of my colleagues, I take a train on the Circle Line to my office at one-north every morning. So on November 5, when my team was given the chance to investigate the cause, I volunteered without hesitation.

From prior investigations by train operator SMRT and the Land Transport Authority (LTA), we already knew that the incidents were caused by some form of signal interference, which led to loss of signals in some trains. The signal loss would trigger the emergency brake safety feature in those trains and cause them to stop randomly along the tracks.

But the incidents — which first happened in August — seemed to occur at random, making it difficult for the investigation team to pinpoint the exact cause.



[Open in app](#)[Get started](#)

- ID of train involved
- Direction of train

We started by cleaning the data. We worked in a [Jupyter Notebook](#), a popular tool for writing and documenting Python code.

As usual, the first step was to import some useful Python libraries.

```
1 import math
2 import xlrd
3 import itertools as it
4 import numpy as np
5 import pandas as pd
6 from datetime import datetime
```

ccl1.py hosted with ❤ by GitHub

[view raw](#)

Snippet 1

We then extracted the useful parts from the raw data.

```
1 dfincidents_0 = pd.read_excel('CCL EVAC E-brake occurrences hourly update_mod.xlsx', sheetname='Au
2 dfincidents_1 = pd.read_excel('CCL EVAC E-brake occurrences hourly update_mod.xlsx', sheetname='No
3
4 # Incident data for Nov had different format
5 dfincidents_1['Time'] = dfincidents_1['Time'].str.strip('hrs').str.strip(' ')
6 dfincidents_1['Time'] = pd.to_datetime(dfincidents_1['Time'], format='%H%M').dt.time
7
8 dfincidents = pd.concat([dfincidents_0, dfincidents_1])
9
10 # Reset the index because they were concatenated from two data sources
11 dfincidents.reset_index(inplace=True, drop=True)
```

ccl2.py hosted with ❤ by GitHub

[view raw](#)

Snippet 2




[Open in app](#)
[Get started](#)

```

2      """
3      Combines the date column and time column into a single column
4      """
5      d = row['Date']
6      t = row['Time']
7
8      return datetime(
9          d.year, d.month, d.day,
10         t.hour, t.minute, t.second
11     )
12
13 # Add DateTime to the data for easier visualization
14 dfincidents['DateTime'] = dfincidents.apply(datetime_from_date_and_time, axis=1)

```

ccl3.py hosted with ❤ by GitHub

[view raw](#)

Snippet 3

This gave us:

	S/N	S/N.1	Date	Traffic Date	PV	Time	Bound	Station from	Station to	Event	Remarks	DateTime
0	1	1	2016-08-28	2016-08-28	PV40	19:32:00	OT	KRG	ONH	EB	point track	2016-08-28 19:32:00
1	2	2	2016-08-28	2016-08-28	PV53	19:39:00	OT	LBD	PPJ	EB	point track	2016-08-28 19:39:00
...	...	...	...	...	...	...	...	...	...	...	...	...
257	57	42	2016-11-04	2016-11-04	PV13	22:29:00	IT	SDM	MBT	EB	NaN	2016-11-04 22:29:00
258	58	43	2016-11-05	2016-11-05	PV43	00:07:00	IT	TLB	HBF	EVAC	Withdrawal train, no pax on-board	2016-11-05 00:07:00

Screenshot 1: Output from initial processing

No clear answers from initial visualisations

We could not find any obvious answers in our initial exploratory analysis, as seen in the



[Open in app](#)[Get started](#)

1. The incidents were spread throughout a day, and the number of incidents across the day mirrored peak and off-peak travel times.

Frequency by Time of Day

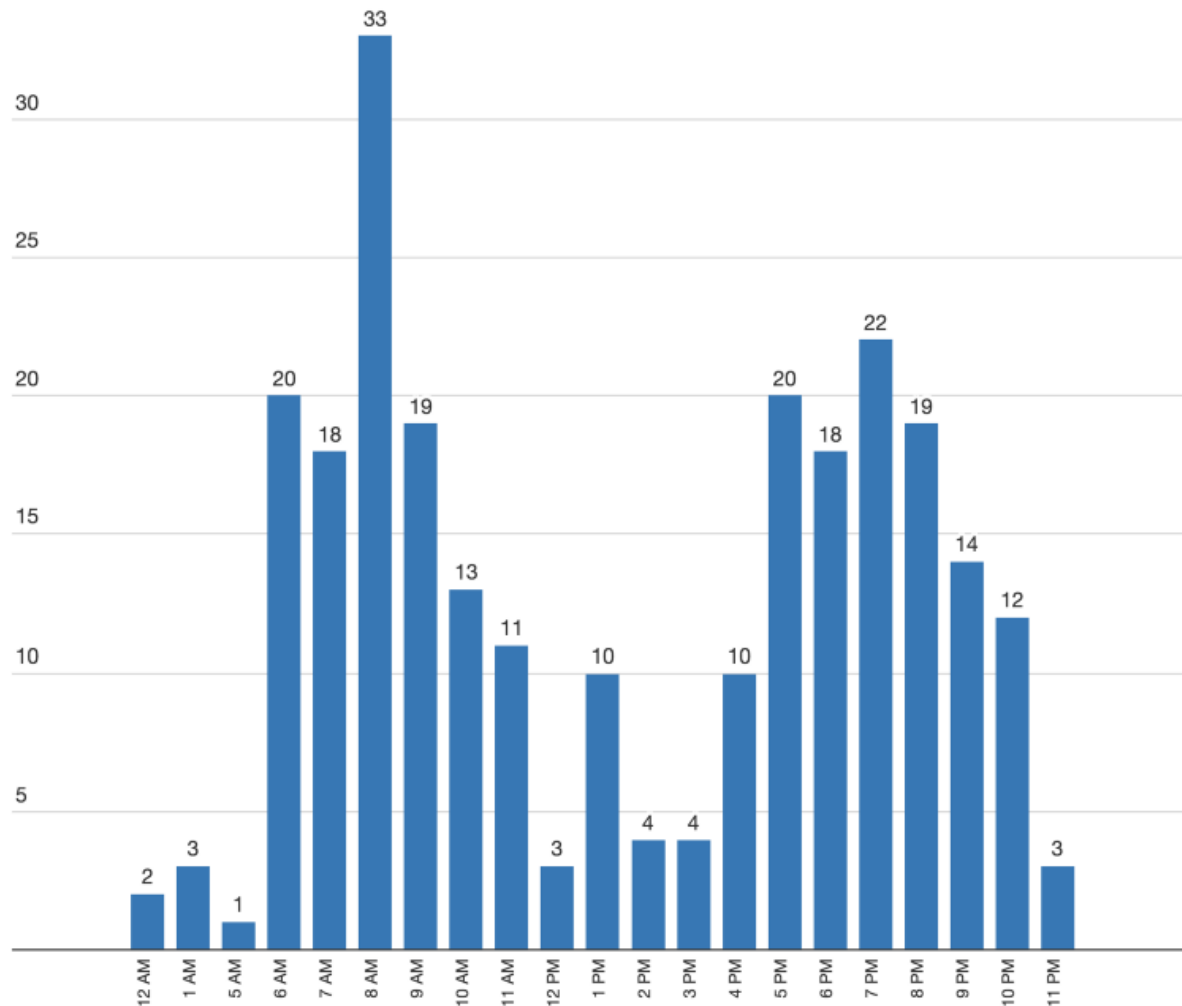


Figure 1: Number of occurrences mirror peak and off-peak travel times.

2. The incidents happened at various locations on the Circle Line, with slightly more occurrences on the west side.




[Open in app](#)
[Get started](#)

Frequency by Station

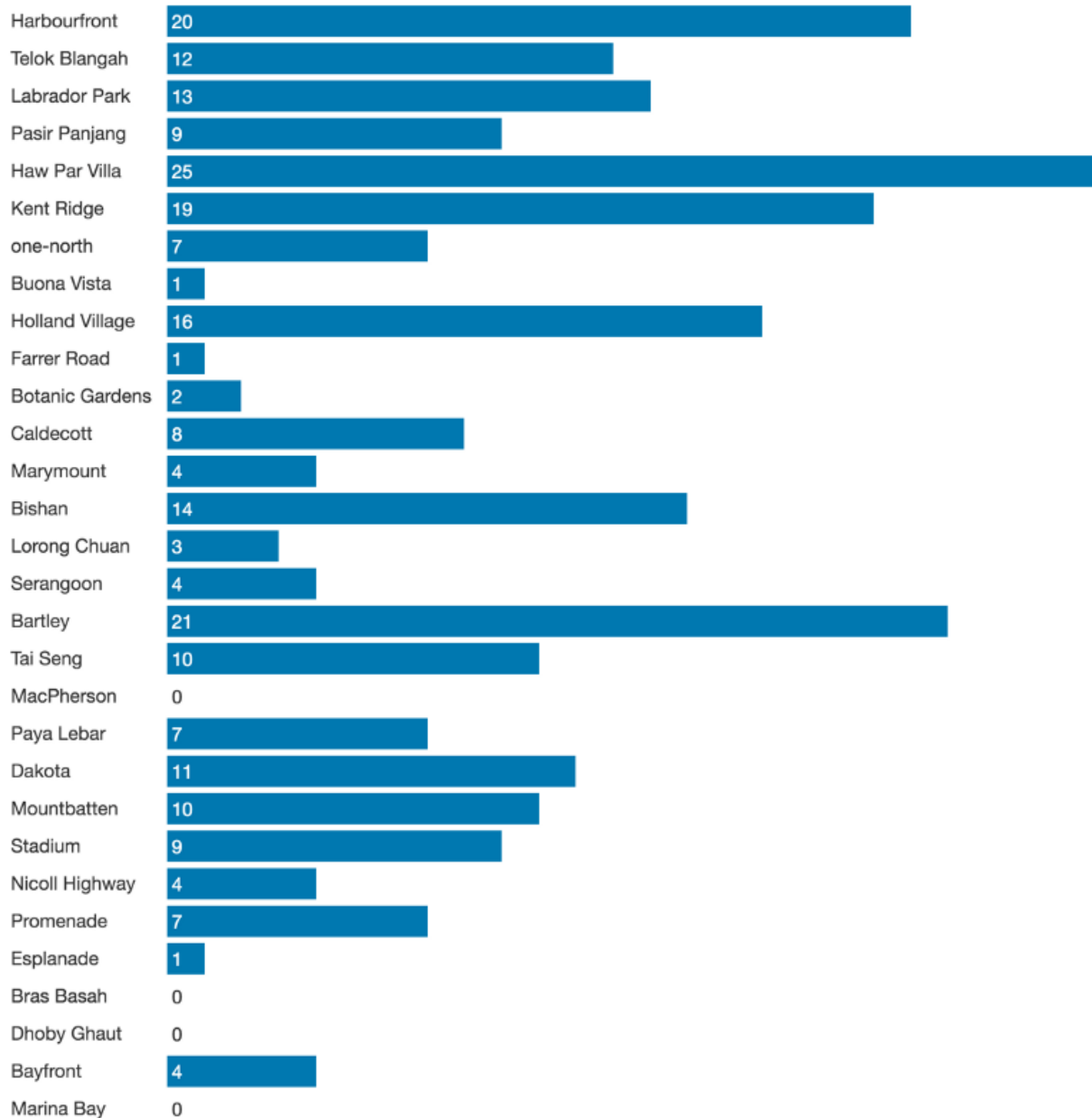
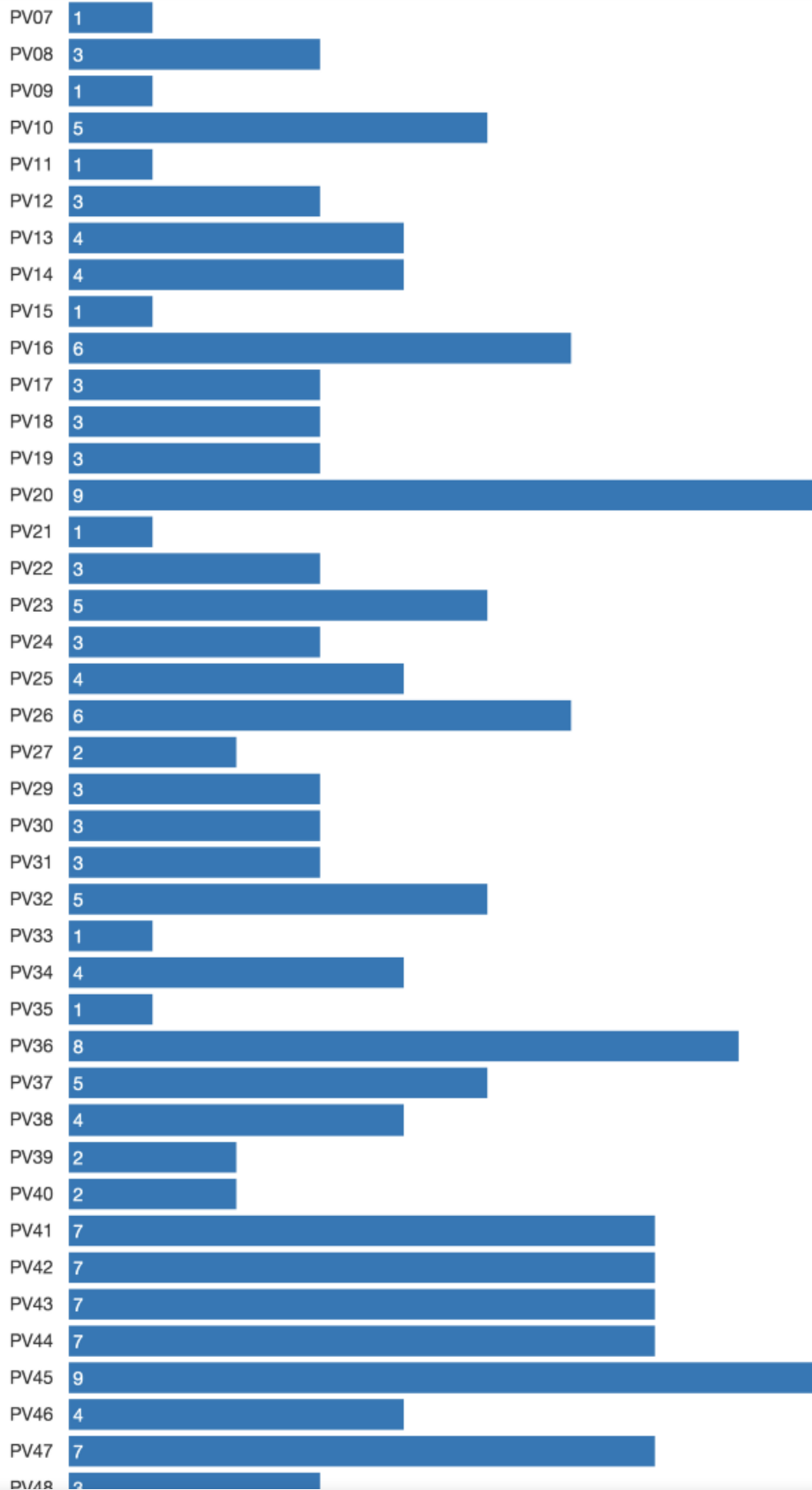


Figure 2: The cause of the interference did not seem to be location-based.

3. The signal interferences did not affect just one or two trains, but many of the trains on the Circle Line. “PV” is short for “Passenger Vehicle”.

Frequency by Train ID



[Open in app](#)[Get started](#)

[Open in app](#)[Get started](#)

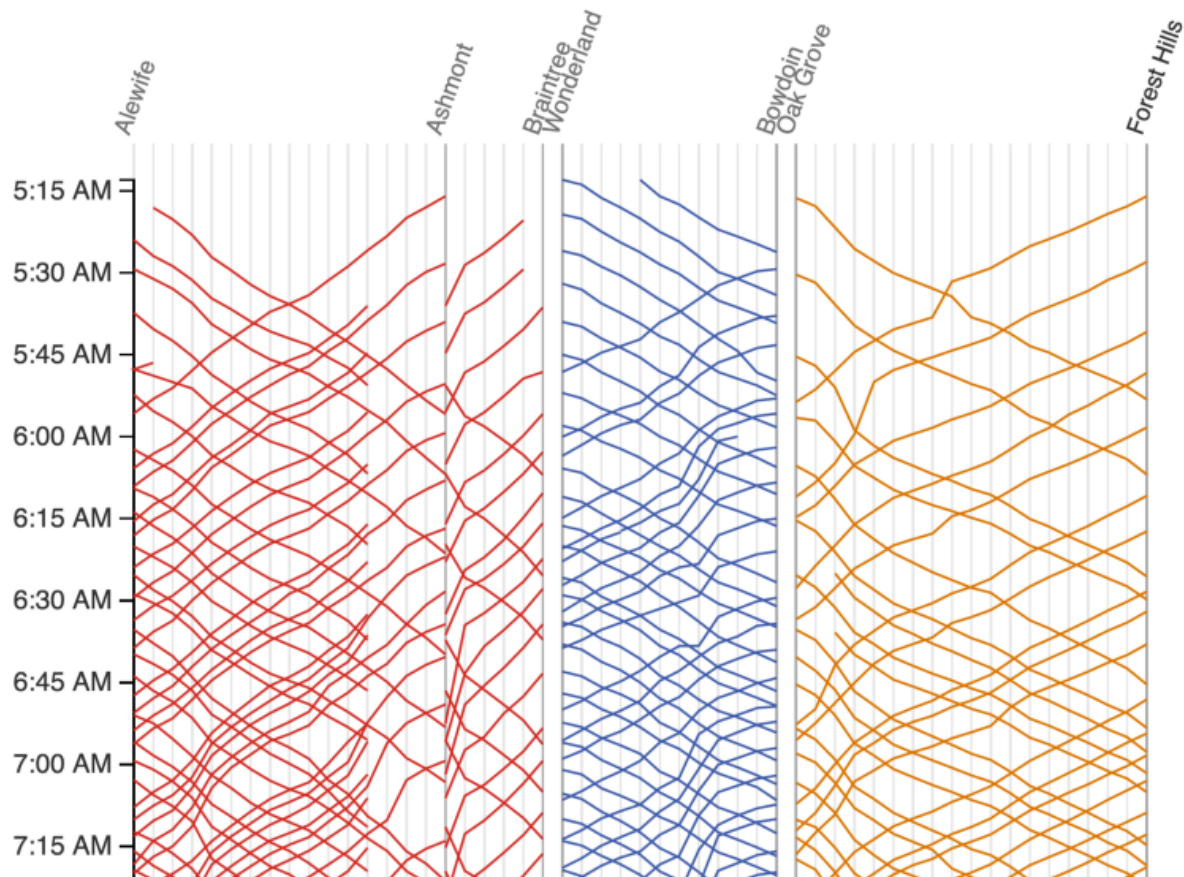
Figure 3: 60 different trains were hit by signal interference.

The Marey Chart: Visualising time, location and direction

Our next step was to incorporate multiple dimensions into the exploratory analysis.

We were inspired by the Marey Chart, which was featured in Edward Tufte's vaunted 1983 classic [The Visual Display of Quantitative Information](#). More recently, it was used by Mike Barry and Brian Card for their [extensive visualisation project](#) on the Boston subway system:




[Open in app](#)
[Get started](#)


Screenshot 2: Taken from <http://mbtaviz.github.io/>

In this chart, the vertical axis represents time — chronologically from top to bottom — while the horizontal axis represents stations along a train line. The diagonal lines represent train movement.

We started by drawing the axes in our version of the Marey Chart:





Open in app

Get started

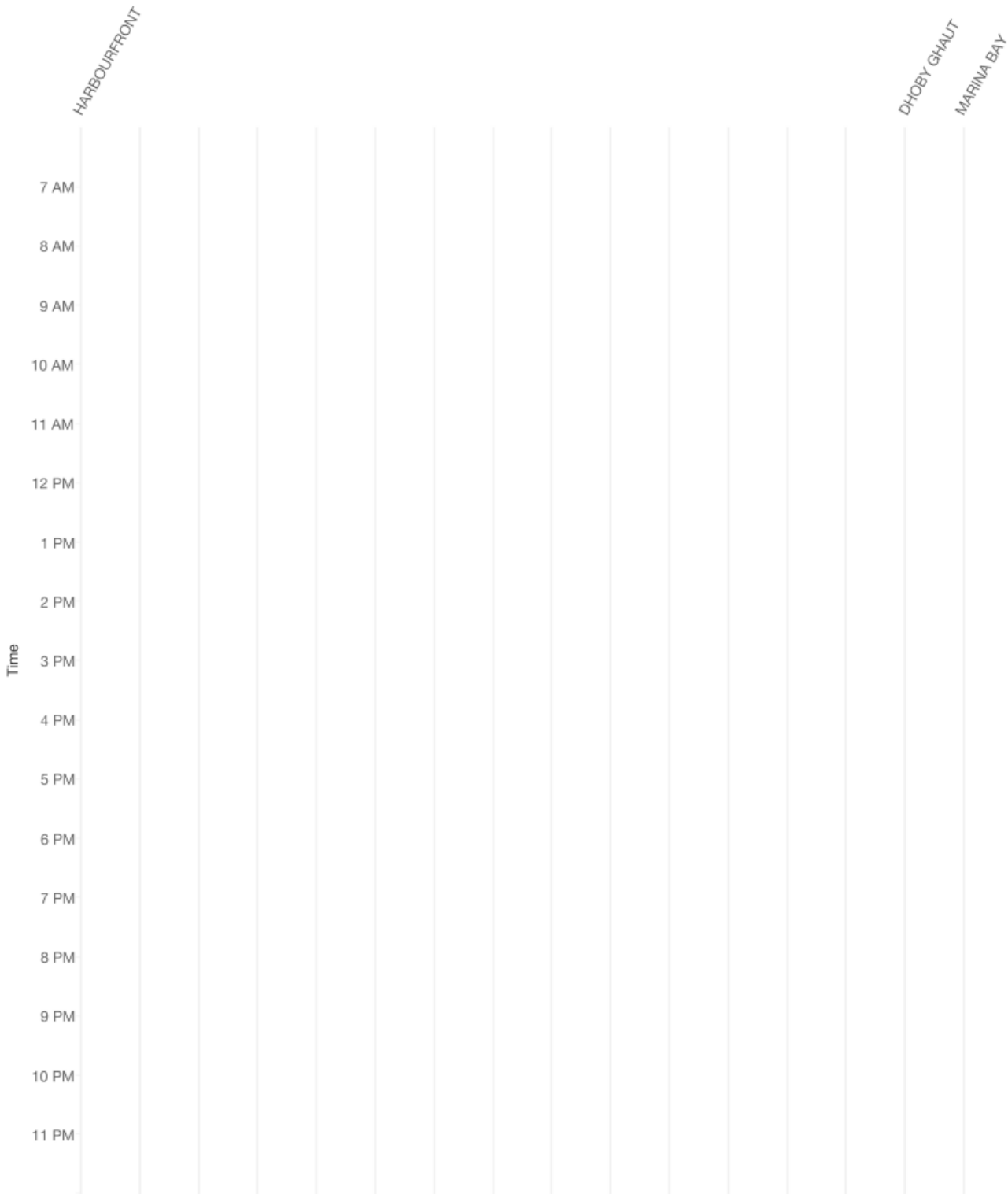


Figure 4: An empty Marey Chart, Circle Line version



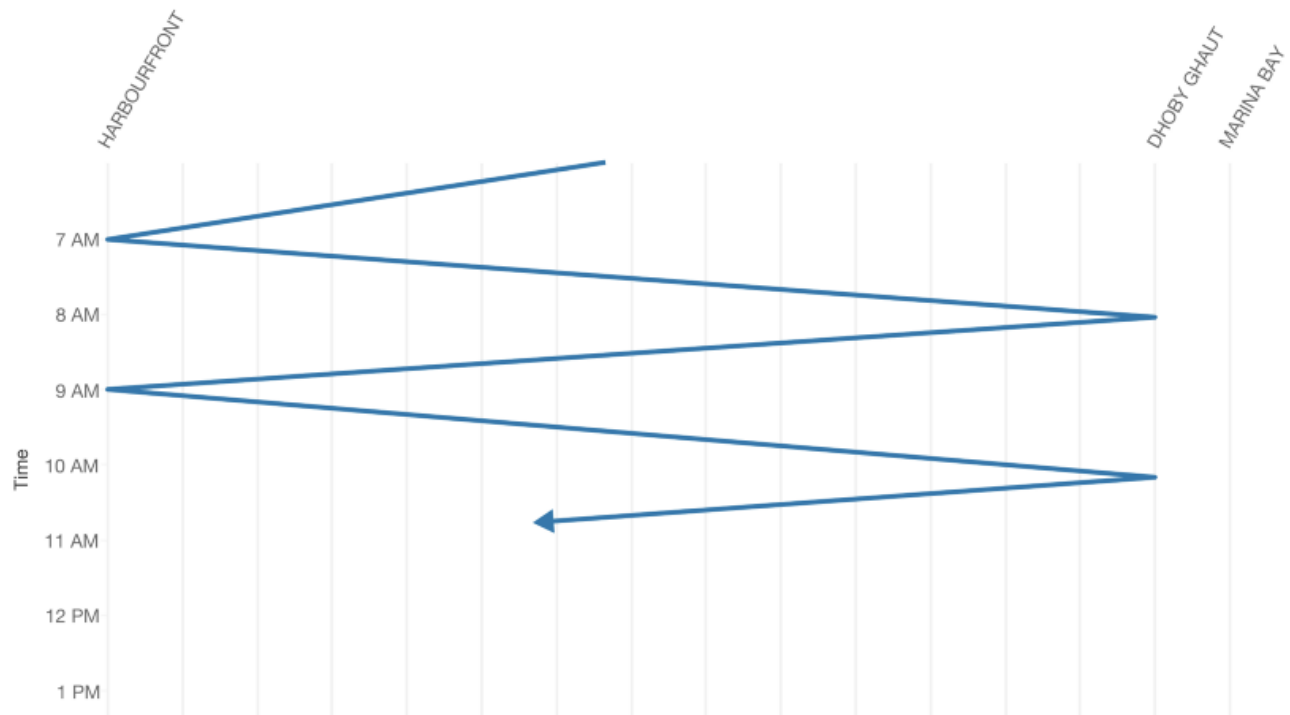

[Open in app](#)
[Get started](#)


Figure 5: Stylised representation of train movement on Circle Line

Our intention was to plot the incidents — which are points instead of lines — on this chart.

Preparing the data for visualisation

First, we converted the station names from their three-letter codes to a number:

- Marina Bay to before Promenade: 0 to 1.5
- Dhoby Ghaut to HarbourFront: 2 to 29

If the incident occurred between two stations, it would be denoted as $0.5 + \text{the lower of the two station numbers}$. For example, If an incident happened between HarbourFront (number 29) and Telok Blangah (number 28), the location would be “28.5”. This made it





[Open in app](#)

[Get started](#)

Snippet 4

And then we computed the numeric location IDs...





[Open in app](#)

[Get started](#)

Snippet 5

And added that to the dataset:




[Open in app](#)
[Get started](#)

Snippet 6

Then we had:

	DateTime	PV	Bound	Station from	Station to	Event	Remarks	LocID
0	2016-08-28 19:32:00	PV40	OT	KRG	ONH	EB	point track	23.5
1	2016-08-28 19:39:00	PV53	OT	LBD	PPJ	EB	point track	26.5
...	...	...	...	...	...	...	...	...
257	2016-11-04 22:29:00	PV13	IT	SDM	MBT	EB	NaN	7.5
258	2016-11-05 00:07:00	PV43	IT	TLB	HBF	EVAC	Withdrawal train, no pax on-board	28.5

Screenshot 3: Output table after location IDs are added

With the data processed, we were able to create a scatterplot of all the emergency braking incidents. Each dot here represents an incident. Once again, we were unable to spot any clear pattern of incidents.





Open in app

Get started

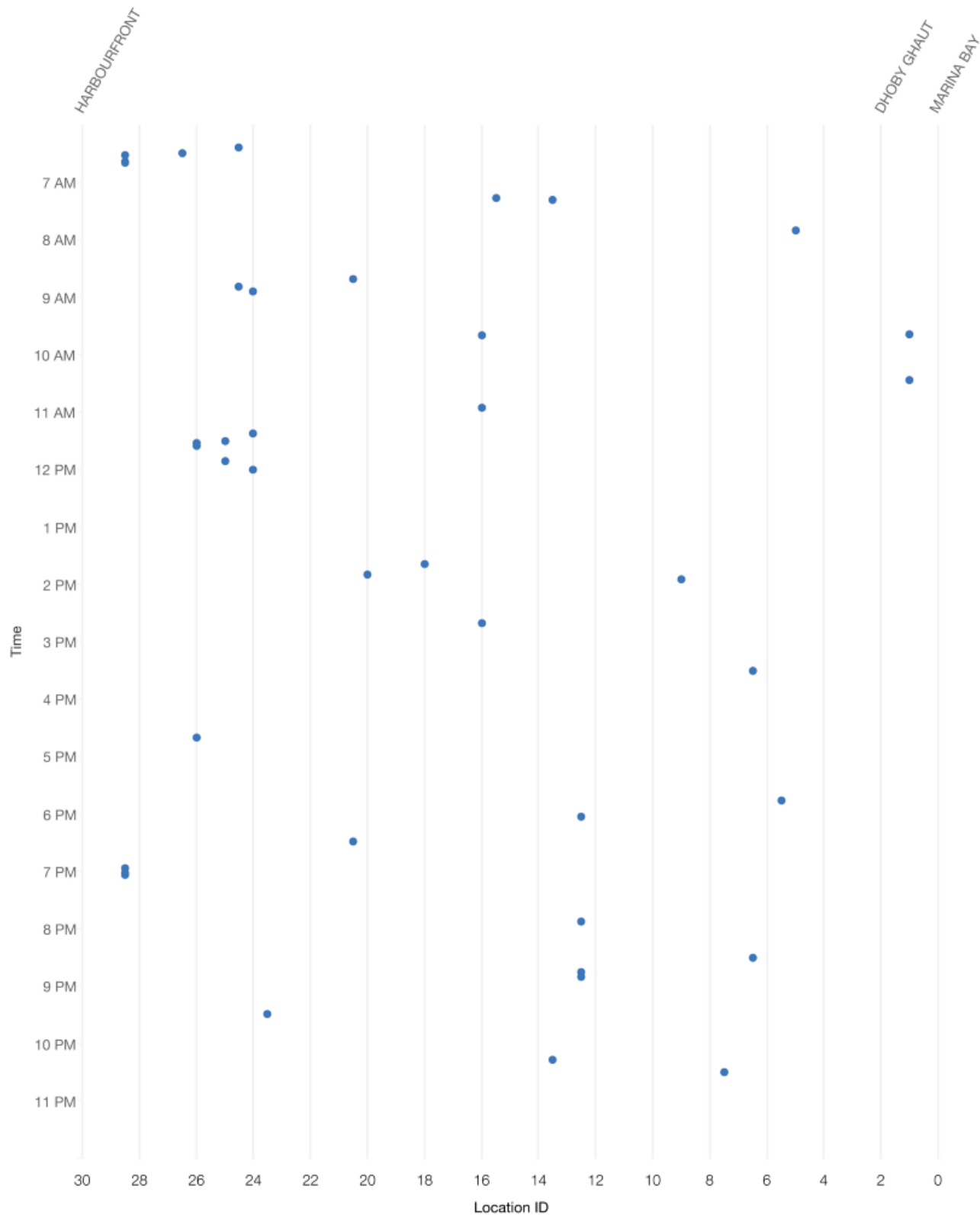


Figure 6: Signal interference incidents represented as a scatterplot





Open in app

Get started

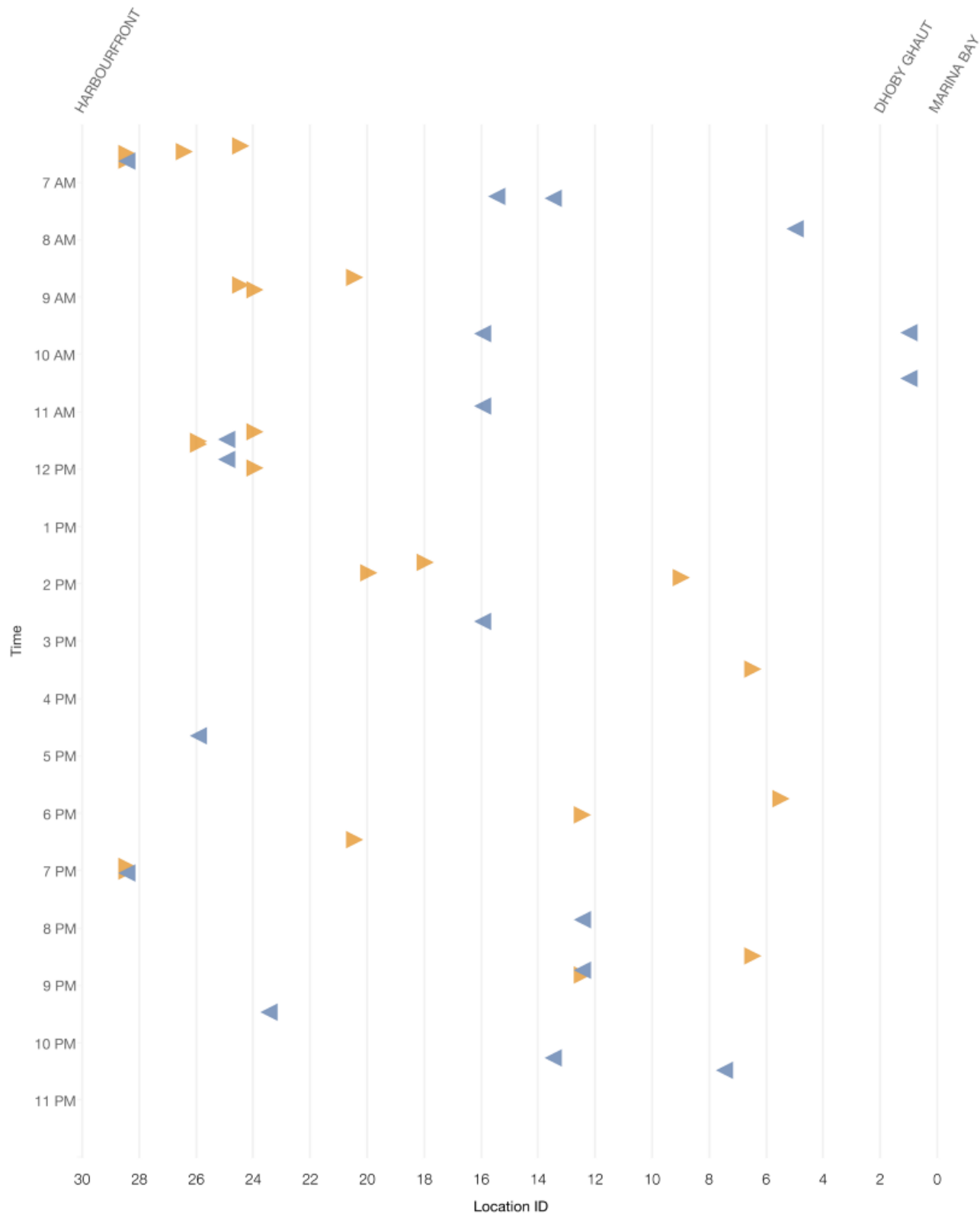


Figure 7: Direction is represented by arrows and colour.



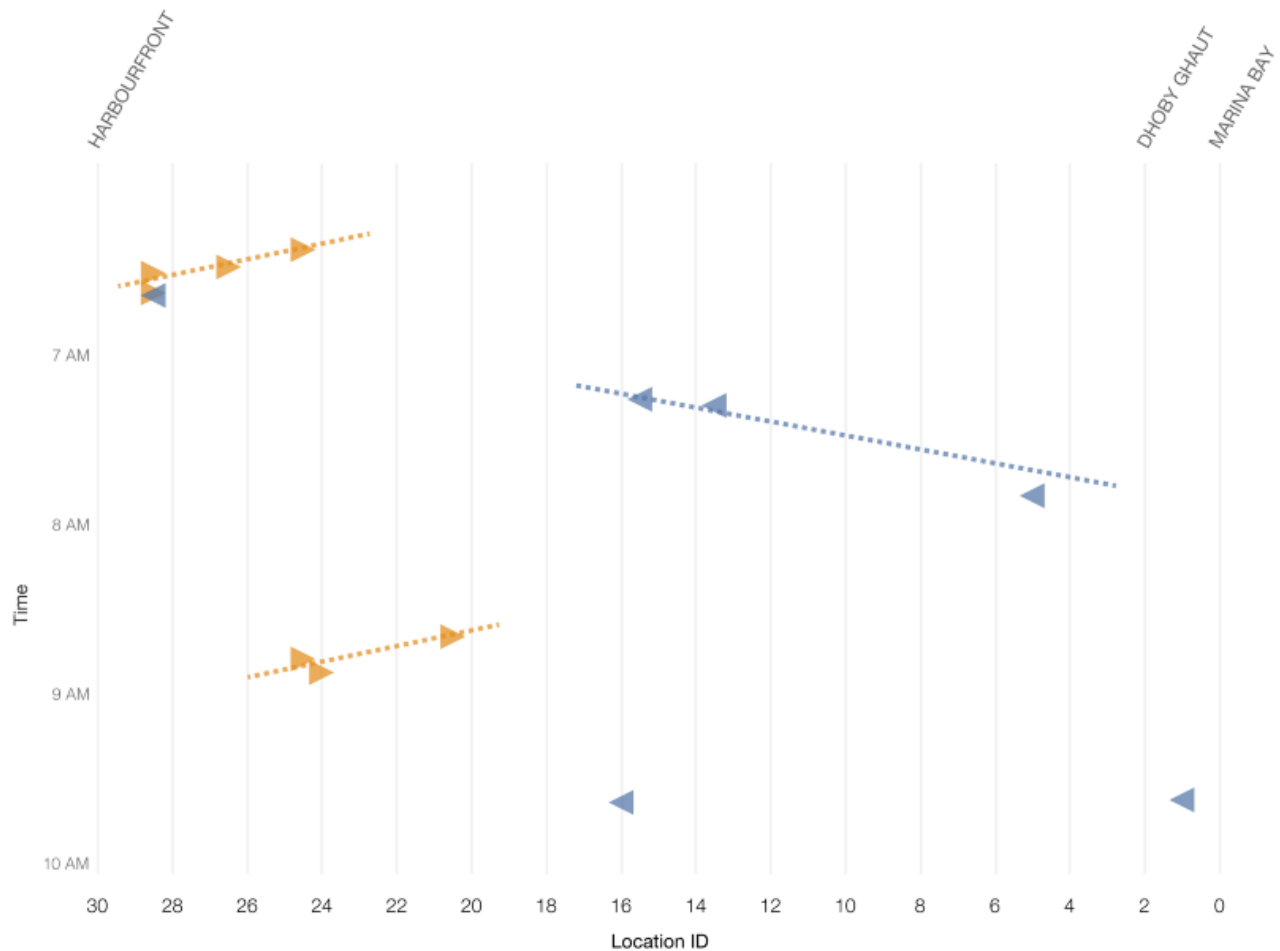

[Open in app](#)
[Get started](#)


Figure 8: Incidents between 6am and 10am

If you read the chart carefully, you would notice that the breakdowns seem to happen *in sequence*. When a train got hit by interference, another train *behind* moving in the same direction got hit soon after.

How can signal interference move through a tunnel?

At this point, it still wasn't clear that a single train was the culprit.

What we'd established was that there seemed to be a pattern over time and location:

Incidents were happening one after another in the opposite direction of the previous




[Open in app](#)
[Get started](#)

Indeed, imaginary lines connecting the incidents looked suspiciously similar to those in a Marey Chart (Screenshot 2). **Could the cause of the interference be a train — in the opposite track?**

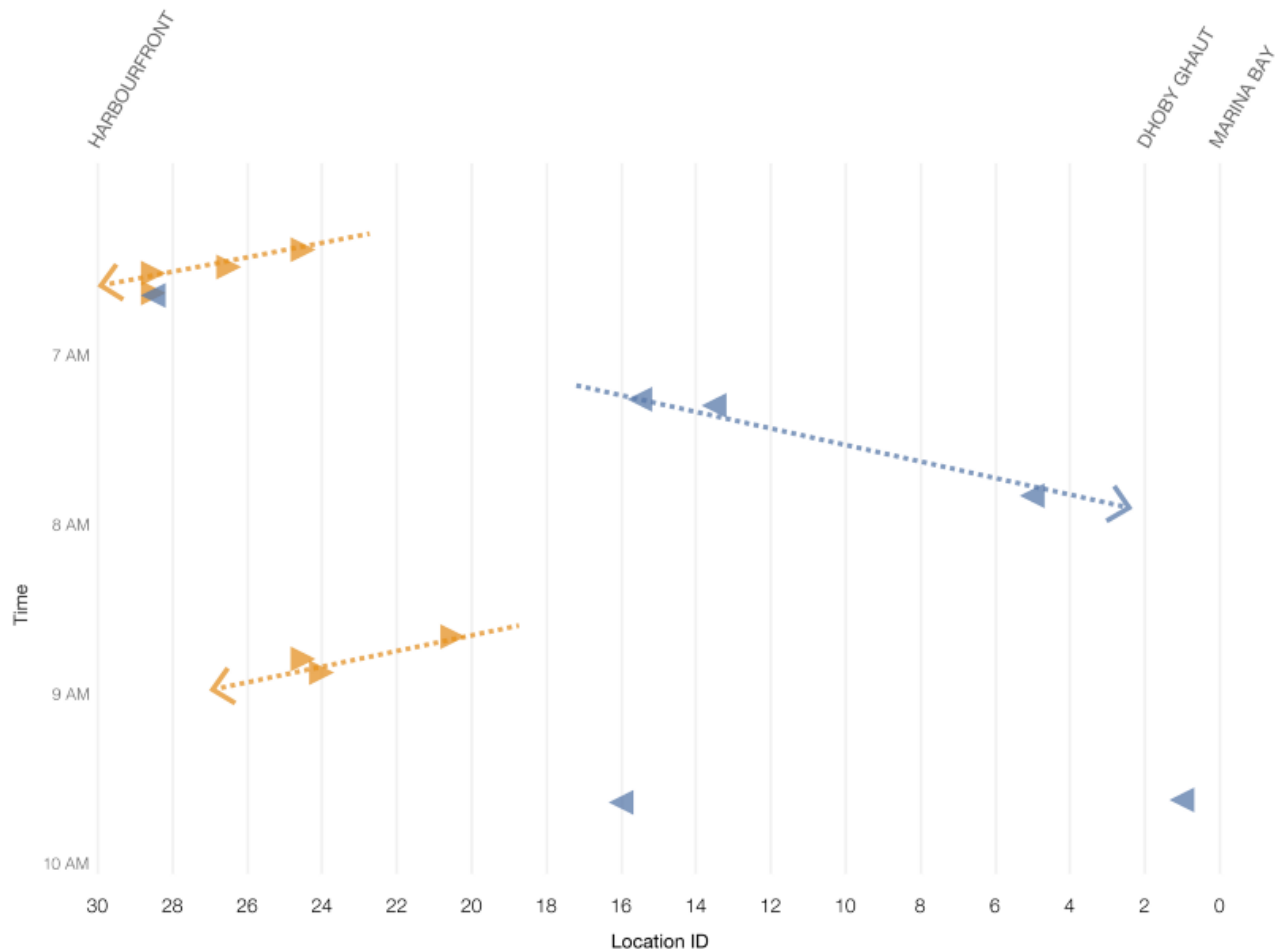


Figure 9: Could it be a train moving in the opposite direction?

We decided to test this “rogue train” hypothesis.

We knew that the travel time between stations along the Circle Line ranges between two and four minutes. This means we could group all emergency braking incidents together if they occur up to four minutes apart.



[Open in app](#)[Get started](#)

Snippet 7

We found all incident pairs that satisfied this condition:



[Open in app](#)[Get started](#)

Snippet 8

We then grouped all related pairs of incidents into larger sets using a disjoint-set data structure. This allowed us to group incidents that could be linked to the same “rogue train”.





[Open in app](#)

[Get started](#)

Snippet 9

Then we applied our algorithm to the data:



[Open in app](#)[Get started](#)

Snippet 10

These were some of the clusters that we identified:

```
[{0, 1},  
 {2, 4},  
 {5, 6, 7},  
 {8, 9},  
 {18, 19, 20},  
 {21, 22, 24, 26, 27},  
 {28, 29, 30, 31, 32, 33, 34},  
 {42, 44, 45},  
 {47, 48},  
 {51, 52, 53, 56}]
```

Next, we calculated the percentage of the incidents that could be explained by our clustering algorithm.



[Open in app](#)[Get started](#)

Snippet 11

The result was:

```
(189, 259, 0.7297297297297297)
```

What it means: **Of the 259 emergency braking incidents in our dataset, 189 cases — or 73% of them — could be explained by the “rogue train” hypothesis.** We felt we were on the right track.

We coloured the incident chart based on the clustering results. Triangles with the same colour are in the same cluster.



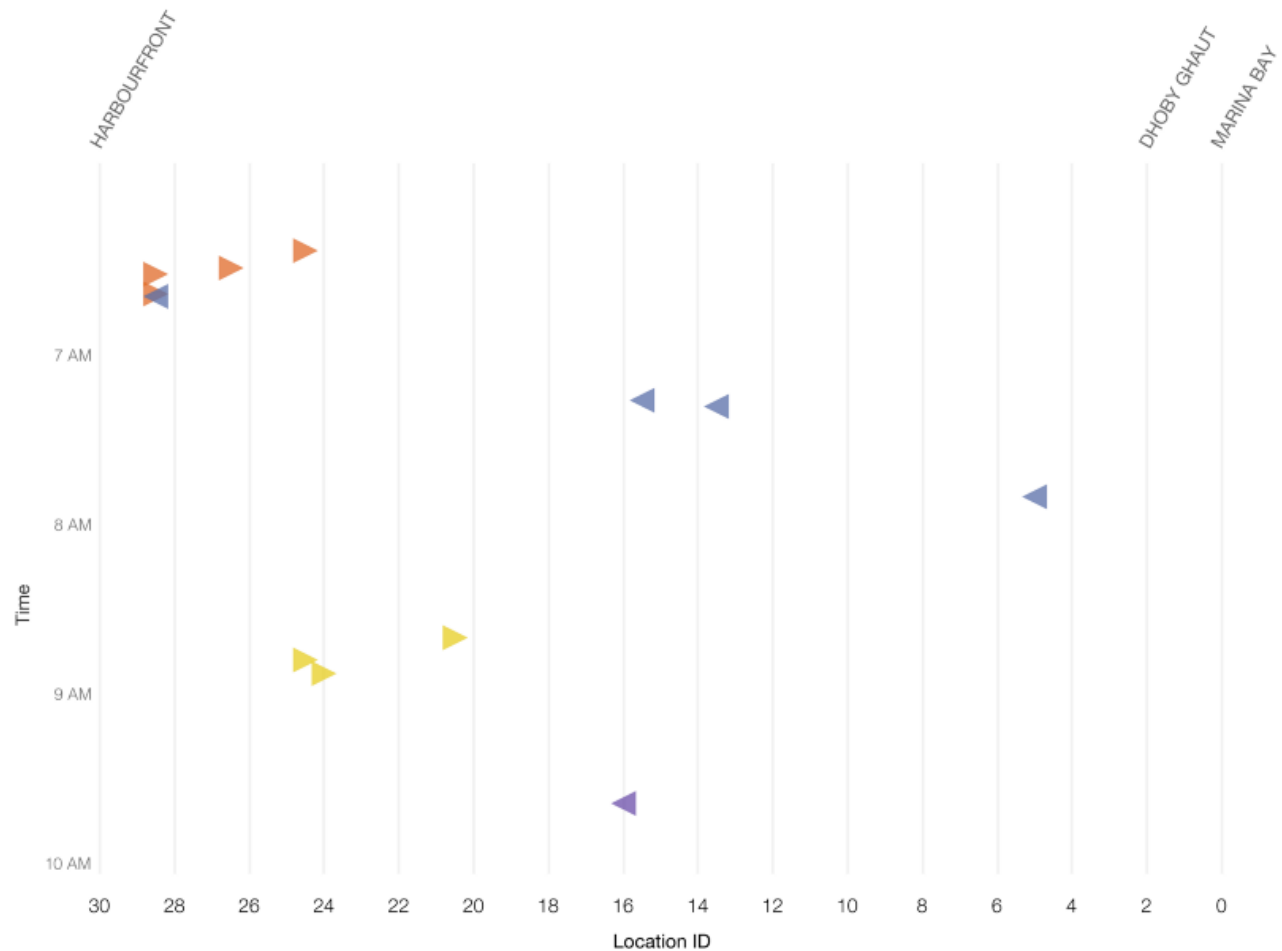
[Open in app](#)[Get started](#)

Figure 10: Incidents clustered by our algorithm

How many rogue trains are there?

As we showed in **Figure 5**, each end-to-end trip on the Circle Line takes about 1 hour. We drew best-fit lines through the incidents plots and the lines closely matched that of **Figure 5**. This strongly implied that there was only one “rogue train”.



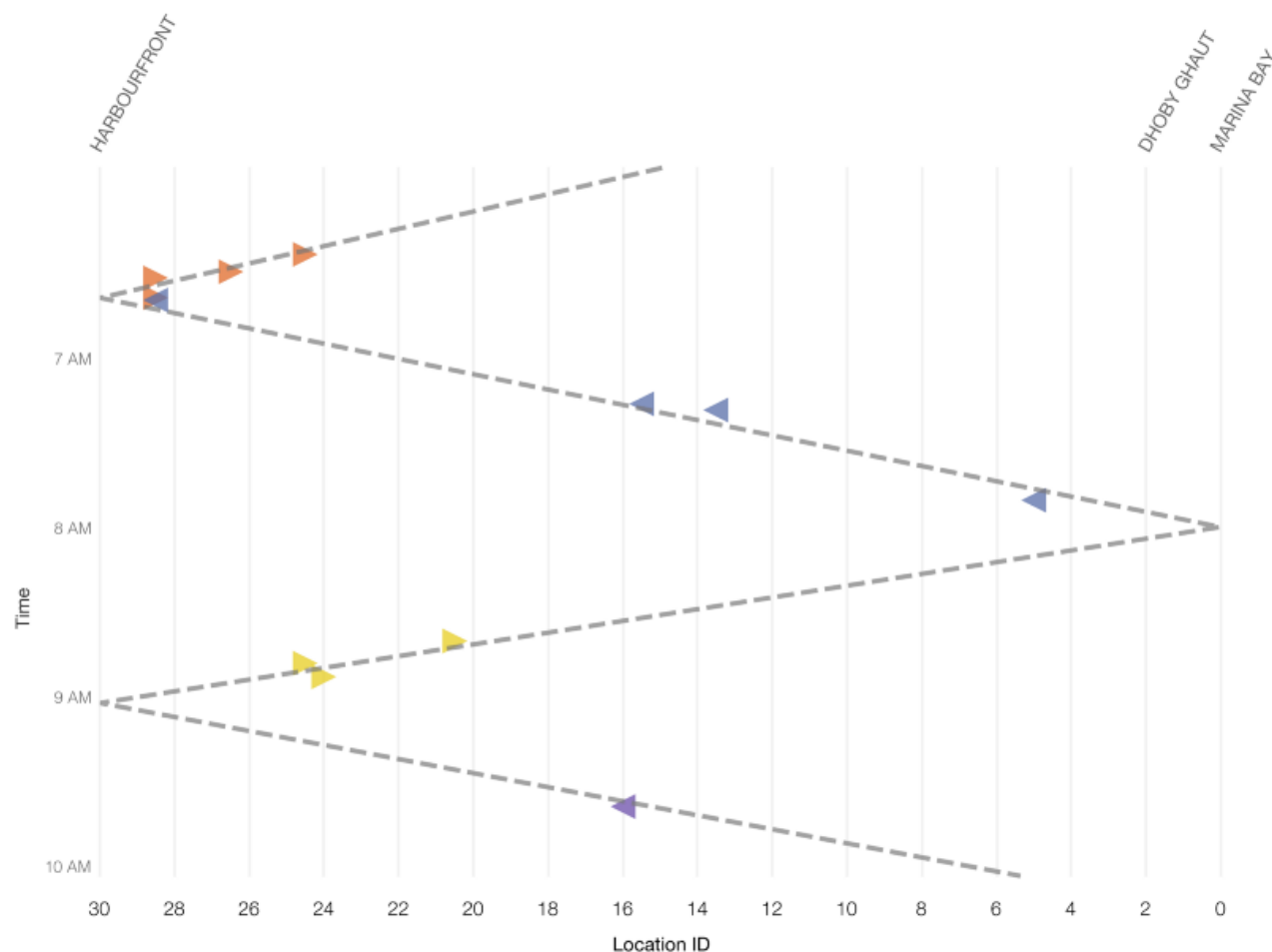

[Open in app](#)
[Get started](#)


Figure 11: Time of clustered incidents strongly implies that the interference could be linked a single train

We also observed that the unidentified “rogue train” itself did not seem to encounter any signalling issues, as it did not appear on our scatter plots.

Convinced that we had a good case, we decided to investigate further.

Catching the rogue train

After sundown, we went to Kim Chuan Depot to identify the “rogue train”. We could not inspect the detailed train logs that day because SMRT needed more time to extract the data. So we decided to identify the train the old school way — by reviewing video records



[Open in app](#)[Get started](#)

At 3am, the team had found the prime suspect: PV46, a train that has been in service since 2015.

Testing the hypothesis

On November 6 (Sunday), LTA and SMRT tested if PV46 was the source of the problem by running the train during off-peak hours. We were right — PV46 indeed caused a loss of communications between nearby trains and activated the emergency brakes on those trains. No such incident happened before PV46 was put into service on that day.

On November 7 (Monday), my team processed the historical location data of PV46 and concluded that more than 95% of all incidents from August to November could be explained by our hypothesis. The remaining incidents were likely due to signal loss that happen occasionally under normal conditions.

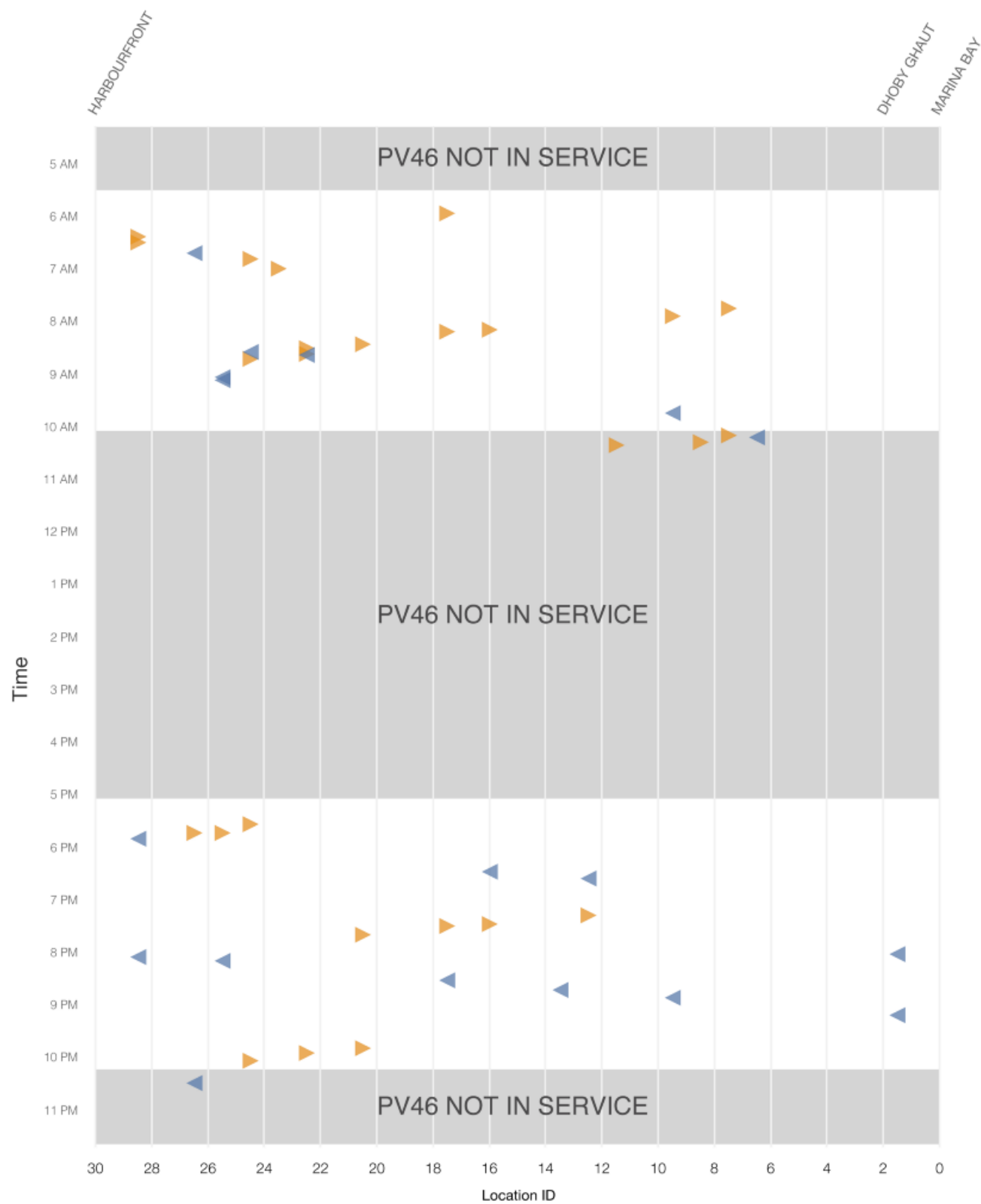
The pattern was especially clear on certain days, like September 1. You can easily see that interference incidents happened during or around the time belts when PV46 was in service.





Open in app

Get started



[Open in app](#)[Get started](#)

Final thoughts

When we first started, my colleagues and I were hoping to find patterns that may be of interest to the cross-agency investigation team, which included many officers at LTA, SMRT and DSTA. The tidy incident logs provided by SMRT and LTA were instrumental in getting us off to a good start, as minimal cleaning up was required before we could import and analyse the data. We were also gratified by the effective follow-up investigations by LTA and DSTA that confirmed the hardware problems on PV46.

From the data science perspective, we were lucky that incidents happened so close to one another. That allowed us to identify both the problem and the culprit in such a short time. If the incidents were more isolated, the zigzag pattern would have been less apparent, and it would have taken us more time — and data — to solve the mystery.

Of course, we were most pleased that all of us can now take the Circle Line to work with confidence again.

Note: The code here was written on November 5, 2016 — the actual day when we were working on SMRT data to identify the cause of the Circle Line incidents. We acknowledge that there could be inefficiencies. You may download a copy of our Jupyter Notebook [here](#).

Daniel Sim, Lee Shangqian and Clarence Ng are data scientists at GovTech's Data Science Division.

Follow Data.gov.sg: [Twitter](#) | [Facebook](#)



[Open in app](#)[Get started](#)

Get an email whenever Open Government Products publishes.

Your email

Subscribe

By signing up, you will create a Medium account if you don't already have one. Review our [Privacy Policy](#) for more information about our privacy practices.

[About](#) [Help](#) [Terms](#) [Privacy](#)

Get the Medium app

